

Quantization

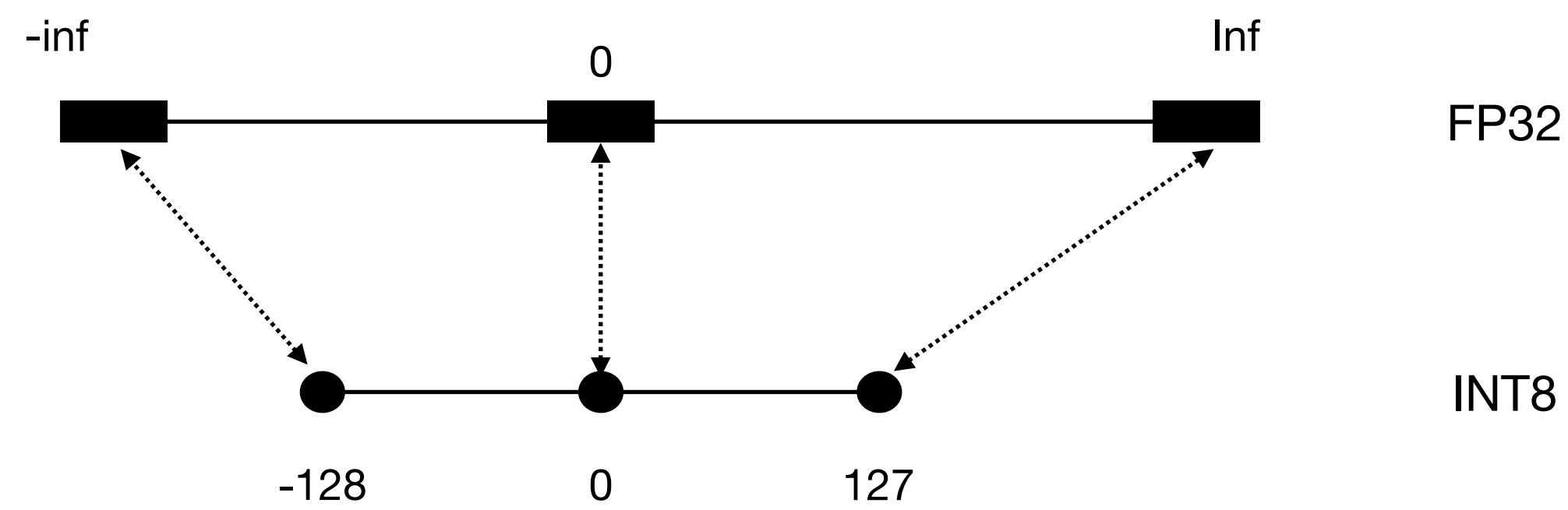
Zhe Li

Why quantization?

- Smaller model size:
 - INT8 cuts model size to 1/4 of FP32.
- Faster inference:
 - Low-precision arithmetic is much faster - especially on GPUs/TPUs that have special INT8/FP16 units.
- Lower power consumption:
 - Higher throughput
- Deployability on edge devices
 - Phone, embedded system ...

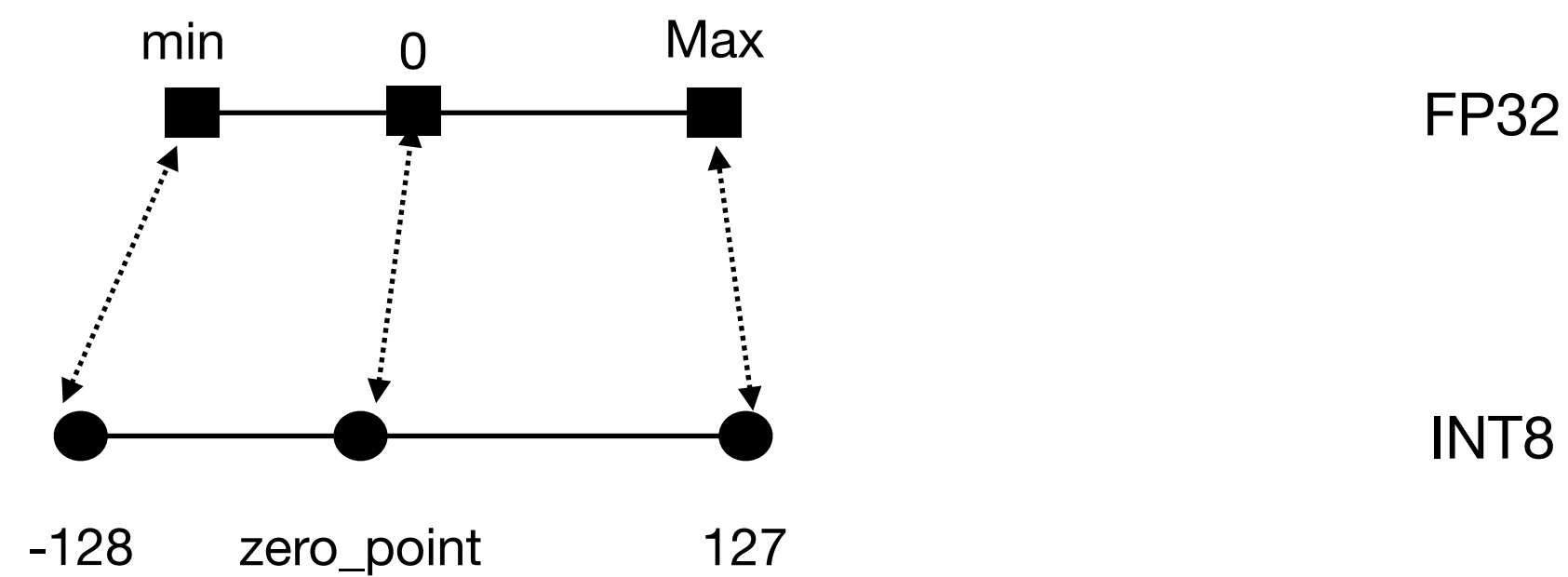
What is quantization?

- Reduce high-precision numerical values (32-bit floats) to low-precision discrete values (like 8-bit or 4-bit integers).
- Using FP32 $\rightarrow$ INT8 as example
 - FP32: $\sim 10^{-45}$ to 10^{38}
 - INT8: $[-128, -127, -126, \dots, 0, 1, \dots, 125, 126, 127]$, total 255 integers



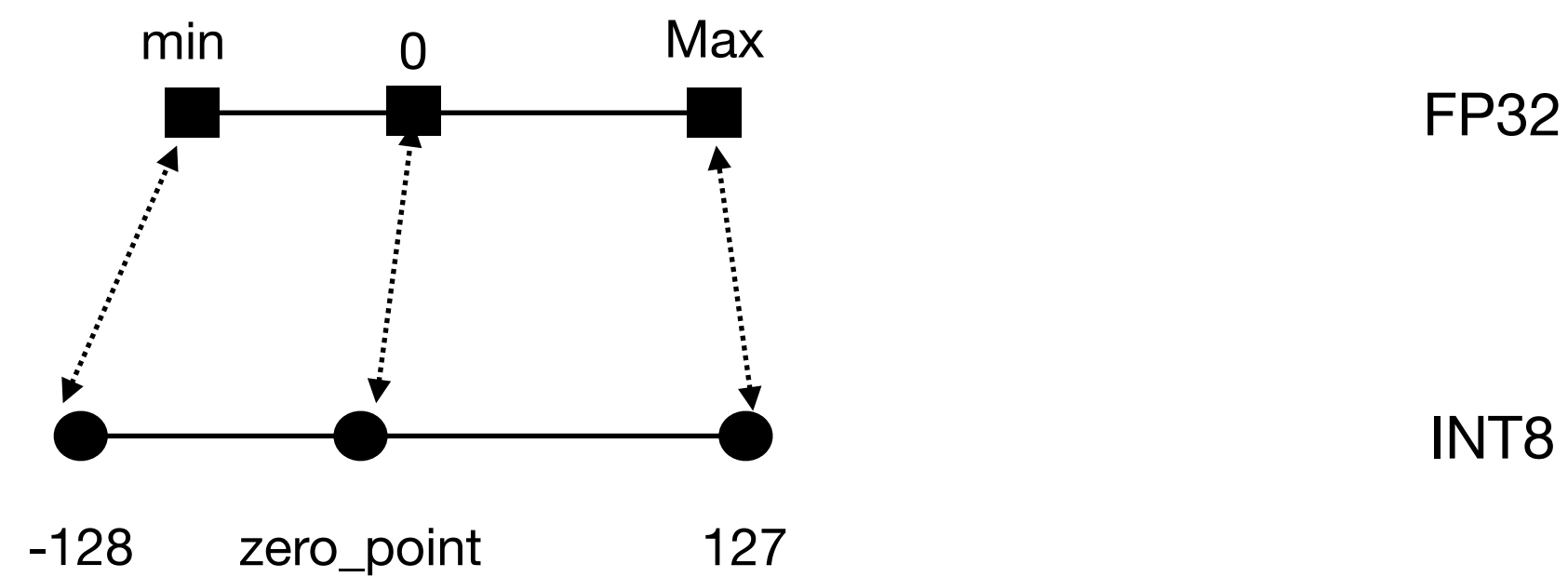
What is quantization?

- Practically, neural network weights and actions don't actually use the full FP32 range.
- In practice, within each layer:
 - Most values are small.
 - Distribution is narrow.



How do quantization?

- How to convert from FP32 to INT8? The following is more general to apply to any high-precision to low-precision conversation.
- Denote x_{float} as FP32, x_{int} as INT8 after quantization.
- Denote x_{minf} and x_{maxf} as min and max of FP32 values observed.
- Denote x_{mini} and x_{maxi} as min and max of INT8, actually -128, 127.



How do quantization?

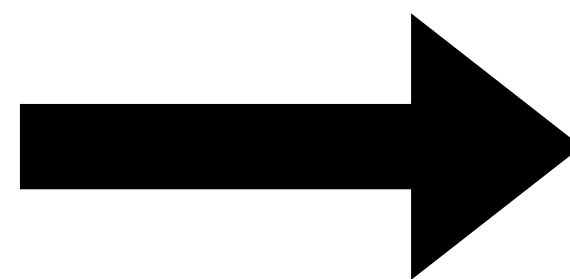
- Define

$$x_{float} = scale * (x_{int} - zero_point)$$

- *scale* and *zero_point* is parameter and will be computed.
- Why can we define it like the above?
 - Affine transformation/mapping (linear + shift).
- How to compute *scale* and *zero_point*?

$$x_{minf} = scale * (x_{minf} - zero_point)$$

$$x_{maxf} = scale * (x_{maxi} - zero_point)$$



$$scale = (x_{maxf} - x_{minf}) / (x_{maxi} - x_{mini})$$

$$zero_point = x_{mini} - \frac{x_{minf}}{scale}$$

How do quantization?

- Define

$$x_{float} = scale * (x_{int} - zero_point)$$

$$scale = (x_{maxf} - x_{minf}) / (x_{maxi} - x_{mini})$$

$$zero_point = x_{mini} - \frac{x_{minf}}{scale}$$

- What are *scale* and *zero_point* exactly?
 - *scale*: how much real FP32 value each integer represents; if *scale* is small, quantization is high-resolution, otherwise, quantization is coarse.
 - *zero_point*: handles shifting to make FP32 0 exactly representable.

$$x_{float} = scale * (x_{int} - zero_point)$$

$$x_{int} = \text{round}\left(\frac{x_{float}}{scale}\right) + zero_point$$

How do quantization? Example

- Given weight with FP32

$$w = \begin{bmatrix} 0.1 & -0.3 \\ 0.5 & -0.2 \end{bmatrix}$$

- Quantizing the above weight to INT8:

- Step 1: find x_{minf} and x_{maxf} , which are -0.3, 0.5

- Step 2: compute $scale = \frac{x_{maxf} - x_{minf}}{x_{maxi} - x_{mini}} = \frac{0.5 + 0.3}{127 + 128} = 0.003137$

- Step 3: compute

$$zero_point = x_{mini} - \frac{x_{minf}}{scale} = round(-128 + \frac{0.3}{0.003137}) = -32$$

How do quantization? Example

- Given weight with FP32

$$w = \begin{bmatrix} 0.1 & -0.3 \\ 0.5 & -0.2 \end{bmatrix}$$

- Quantizing the above weight to INT8:

- Step 4: quantizing every FP32 number $x_{int} = \text{round}(\frac{x_{float}}{scale}) + zero_point$

- After quantization, we have INT8 weight:

$$w_{INT8} = \begin{bmatrix} 0 & -128 \\ 127 & -96 \end{bmatrix}$$

How do quantization?

Weights in Original neural network

$$W = \begin{bmatrix} 0.1 & -0.3 \\ 0.5 & -0.2 \end{bmatrix}$$

Quantization Weights in Original neural network

$$W_{INT8} = \begin{bmatrix} 0 & -128 \\ 127 & -96 \end{bmatrix}$$

$$scale = 0.003137$$

$$zero_point = -32$$

- This quantization procedure could be applied per tensor, per channel, per any granularity.

How do quantization?

- Previously we discussed quantization on weights, how does it works in reality with input x ?

$$y = Wx$$

- Apply exact same procedure on input x ? In practices, we collect the representative input data to compute *scale* and *zero_point* for input

$$W_{INT8} = \begin{bmatrix} 0 & -128 \\ 127 & -96 \end{bmatrix}$$

$$scale_W = 0.003137$$

$$zero_point_W = -32$$

$$x_{INT8} = \begin{bmatrix} 76 & -126 \\ 53 & -87 \end{bmatrix}$$

$$scale_x = 0.05137$$

$$zero_point_x = -18$$

How do quantization?

- Integer matrix multiplication

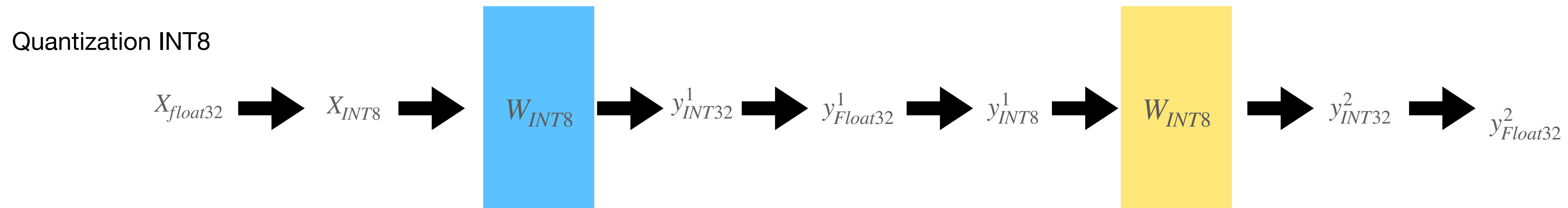
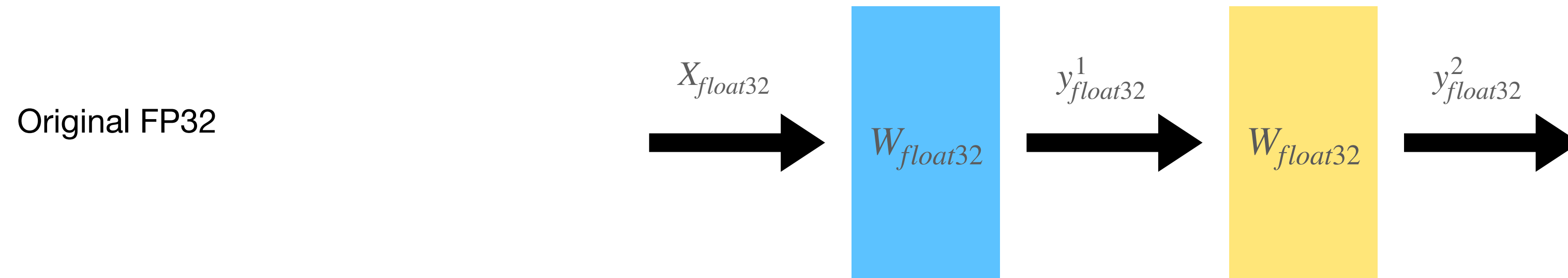
$$y_{INT32} = (W_{INT8} - zero_point_W) * (x_{INT8} - zero_point_x)$$

- Mostly, the above is conducted by INT32 accumulation for INT8*INT8.

$$y_{float32} = scale_W * scale_x * y_{INT32}$$

- Can apply the same quantization procedure on $y_{float32}$, which is activation quantization.

How do quantization?



- The above shows the entire procedure from input FP32 to output FP32 through two layer networks.

Quantization Aware Training (QAT)

- So far, it is all about Post-Training Quantization (PTQ), which may have severe performance drop.
- QAT simulates INT8 inference during training so that the model learns to compensate for quantization error.
- How it works:
 - During forward pass: insert fake quantizers that mimic INT8 quantization.
 - Using float operation but simulating INT8 operation.
 - Does not actually convert parameters to INT8, the value stay float32 for gradient updates
 - Allows network to “see” quantization errors during training.

$$x_q = \text{round}\left(\frac{x}{scale}\right) * scale$$

Quantization Aware Training (QAT)

- QAT simulates INT8 inference during training so that the model learns to compensate for quantization error.
- How it works:
 - During forward pass: insert fake quantizers that mimic INT8 quantization.

$$x_q = \text{round}\left(\frac{x}{scale}\right) * scale$$

- How to pick *scale*:
 - For symmetric quantization $scale = \frac{\max(|x_{minf}|, |x_{maxf}|)}{2^{b-1} - 1}$, b is the number of bits,
 x_{minf} and x_{maxf} observed min and max of the tensor.
 - Learned Step Size Quantization (LSQ)

Quantization Aware Training (QAT)

- QAT simulates INT8 inference during training so that the model learns to compensate for quantization error.
- How it works:
 - During backward pass: insert fake quantizers that mimic INT8 quantization.
 - Gradients bypass rounding using the Straight-Through Estimator (STE).

$$\frac{\partial x_q}{\partial x} \approx 1$$

Lower-Bit Quantization

- Goal: Go Beyond INT8 to INT4, INT2 and even 1-bit (binary/ternary) weights and activations.
- Challenges:
 - Accuracy drop is severe in very deep models (LLMs).
 - Training becomes unstable.
- Potential Remedies:
 - Mixed precision quantization (Different layers have different bit-widths).
 - Adaptive quantization: learn per-weight or per-channel bit-widths.
 - Advanced rounding/stochastic quantization schemes.

Lower-Bit Quantization

- Goal: Go Beyond INT8 to INT4, INT2 and even 1-bit (binary/ternary) weights and activations.
- Challenges:
 - Accuracy drop is severe in very deep models (LLMs).
 - Training becomes unstable.
- Potential Remedies:
 - Mixed precision quantization (Different layers have different bit-widths).
 - Adaptive quantization: learn per-weight or per-channel bit-widths.
 - Advanced rounding/stochastic quantization schemes.